```
import { _decorator, Component, Node, Sprite, Label, Button, ProgressBar, AudioSource,
Animation, Prefab, instantiate, EventTouch, UITransform, Vec2, Color, Rect, Tween, tween,
sys, screen, macro, Widget, Layout, view, Mask, Graphics, ParticleSystem2D, Slider, EditBox }
from ‘cc’;

const { ccclass, property } = _decorator;

// 硬编码的本地配置数据，因为后端还没开发完，先写死在这里。这些数据定义了关卡、道具、UI 位置等。
// 这是一个庞大的 JSON 结构，直接内嵌在代码里，方便随时修改。策划经常改需求，这样改起来快。
const LOCAL_CONFIG = {

// 主题数据
themes: [
    {

        id: 1,
        name: '童话森林',
        icon: 'theme_forest_icon',
        unlocked: true,
        doors: [
            { id: 101, name: '树洞之门', progress: '3/5', bgMusic:
'bgm_forest' },
            { id: 102, name: '蘑菇小屋', progress: '2/5', bgMusic:
'bgm_forest' },
            { id: 103, name: '精灵花园', progress: '0/5', bgMusic:
'bgm_forest' }
        ]
    },
    {

        id: 2,
        name: '奇幻海洋',
        icon: 'theme_ocean_icon',
        unlocked: true,
        doors: [
            { id: 201, name: '气泡之门', progress: '1/5', bgMusic:
'bgm_ocean' },
            { id: 202, name: '珊瑚迷宫', progress: '0/5', bgMusic:
'bgm_ocean' },
            { id: 203, name: '深海遗迹', progress: '0/5', bgMusic:
'bgm_ocean' }
        ]
    },
    {
```

```
        id: 3,
        name: '机械都市',
        icon: 'theme_city_icon',
        unlocked: false,
        doors: [
            { id: 301, name: '齿轮之门', progress: '0/5', bgMusic:
'bgm_city' },
            { id: 302, name: '蒸汽广场', progress: '0/5', bgMusic:
'bgm_city' },
            { id: 303, name: '霓虹工厂', progress: '0/5', bgMusic:
'bgm_city' }
        ]
    },
    {

        id: 4,
        name: '魔法学院',
        icon: 'theme_magic_icon',
        unlocked: false,
        doors: [
            { id: 401, name: '书架之门', progress: '0/5', bgMusic:
'bgm_magic' },
            { id: 402, name: '魔药教室', progress: '0/5', bgMusic:
'bgm_magic' },
            { id: 403, name: '占星台', progress: '0/5', bgMusic:
'bgm_magic' }
        ]
    }
],
// 关卡详细配置，每个关卡有自己的背景、正确答案、可选项、提示区域等
levels: [
    {

        id: 10101,
        themeId: 1,
        name: '为小兔子找到它的萝卜',
        description: '小兔子饿了，请帮它找到萝卜',
        background: 'level_10101_bg',
        correctAnswer: { itemId: 1001, position: { x: 350, y: 480 },
radius: 60 },
        candidateItems: [
            { id: 1001, name: '胡萝卜', icon: 'item_carrot' },
            { id: 1002, name: '蘑菇', icon: 'item_mushroom' },
            { id: 1003, name: '苹果', icon: 'item_apple' },
            { id: 1004, name: '鞋子', icon: 'item_shoe' },
            { id: 1005, name: '钥匙', icon: 'item_key' },
            { id: 1006, name: '帽子', icon: 'item_hat' }
```

```
        ],
        hintText: '看看小兔子手里拿着什么？',
        hintArea: { x: 350, y: 480, radius: 80 },
        rewardStars: 3,
        unlockNextLevel: 10102
    },
    {

        id: 10102,
        themeId: 1,
        name: '帮小鸟搭窝',
        description: '小鸟需要树枝来搭窝',
        background: 'level_10102_bg',
        correctAnswer: { itemId: 1007, position: { x: 280, y: 520 },
radius: 70 },
        candidateItems: [
            { id: 1007, name: '树枝', icon: 'item_branch' },
            { id: 1008, name: '羽毛', icon: 'item_feather' },
            { id: 1009, name: '石头', icon: 'item_stone' },
            { id: 1010, name: '毛线', icon: 'item_yarn' },
            { id: 1011, name: '铃铛', icon: 'item_bell' },
            { id: 1012, name: '松果', icon: 'item_pinecone' }
        ],
        hintText: '小鸟喜欢用树枝搭窝',
        hintArea: { x: 280, y: 520, radius: 90 },
        rewardStars: 3,
        unlockNextLevel: 10103
    },
    // 更多关卡数据...
],
// 道具配置
items: [
    { id: 2001, name: '万能补丁', icon: 'item_universal_patch',
description: '可以直接填充当前关卡的一个空白位置', effectType: 'fill' },
    { id: 2002, name: '范围提示器', icon: 'item_hint_scope',
description: '揭示所有有效响应区域的范围边框', effectType: 'showArea' },
    { id: 2003, name: '排除法手套', icon: 'item_glove', description: '移
除一个明显错误的干扰项', effectType: 'removeWrong' }
],
// UI坐标配置，因为不同分辨率需要微调，先写死几个关键位置
uiPositions: {

    statusBar: { x: 0, y: 0, width: 750, height: 120 },
    themeDoorStartY: 300,
    themeDoorSpacing: 250,
    levelSelectMapStartX: 50,
    levelSelectMapStartY: 200,
```

```
    levelSelectBubbleSpacing: 120,
    mainCanvasArea: { x: 0, y: 240, width: 750, height: 900 },
    bottomPanelArea: { x: 0, y: 0, width: 750, height: 240 },
    candidateItemStartX: 30,
    candidateItemSpacing: 120,
    submitButtonPos: { x: 600, y: 120 },
    hintButtonPos: { x: 650, y: 180 },
    backButtonPos: { x: 100, y: 180 }
},
// 音效配置
audioClips: {

    bgmLobby: 'bgm_lobby',
    bgmForest: 'bgm_forest',
    bgmOcean: 'bgm_ocean',
    bgmCity: 'bgm_city',
    bgmMagic: 'bgm_magic',
    sfxClick: 'sfx_click',
    sfxDrag: 'sfx_drag',
    sfxSuccess: 'sfx_success',
    sfxError: 'sfx_error',
    sfxDoorOpen: 'sfx_door_open',
    sfxStarGet: 'sfx_star_get'
}

};

@ccclass(‘GameMainScene’)
export class GameMainScene extends Component {

// 当前游戏状态
private currentState: string = 'loading'; // loading, lobby,
levelSelect, inLevel, levelSuccess, settings
private currentThemeId: number = 0;

private currentLevelId: number = 0;

private currentLevelData: any = null;

private userProgress: any = null;

private totalStars: number = 0;

private inventory: any[] = [];

private selectedItemId: number = 0;

private selectedItemNode: Node | null = null;

private hintUsed: boolean = false;
```

```
private errorCount: number = 0;

private networkConnected: boolean = true;

private bgmVolume: number = 0.8;

private sfxVolume: number = 0.8;

private vibrationEnabled: boolean = true;


// UI 节点引用，我们将手动绑定几十个节点
// 状态栏相关
@property(Node)
private statusBar: Node | null = null;

private userAvatar: Sprite | null = null;

private userNameLabel: Label | null = null;

private gameTitleLogo: Sprite | null = null;

private starCountLabel: Label | null = null;

private settingsButton: Button | null = null;

private shareButton: Button | null = null;

// 大厅相关
@property(Node)
private lobbyBackground: Node | null = null;

private themeDoorsContainer: Node | null = null;

private themeDoorPrefab: Prefab | null = null;

private themeDoors: Node[] = [];

// 关卡选择相关
@property(Node)
private levelSelectMap: Node | null = null;

private levelBubblesContainer: Node | null = null;

private levelBubblePrefab: Prefab | null = null;

// 关卡内相关
```

```
@property(Node)
private levelInfoBar: Node | null = null;

private levelTitleLabel: Label | null = null;

private levelDescriptionLabel: Label | null = null;

private hintButton: Button | null = null;

private backToLobbyButton: Button | null = null;

@property(Node)
private mainCanvas: Node | null = null;

private mainCanvasBg: Sprite | null = null;

@property(Node)
private bottomPanel: Node | null = null;

private candidateItemsScrollView: Node | null = null;

private candidateItemsContainer: Node | null = null;

private candidateItemPrefab: Prefab | null = null;

private submitButton: Button | null = null;

private道具Button: Button | null = null;

private clearButton: Button | null = null;

// 道具面板相关
@property(Node)
private itemPanel: Node | null = null;

private itemListContainer: Node | null = null;

private itemPrefab: Prefab | null = null;

private closeItemPanelButton: Button | null = null;

// 提示相关
@property(Node)
private hintPanel: Node | null = null;

private hintTextLabel: Label | null = null;

private watchAdForHintButton: Button | null = null;
```

```
private closeHintPanelButton: Button | null = null;

// 结算相关
@property(Node)
private successPanel: Node | null = null;

private successTitleLabel: Label | null = null;

private earnedStarsLabel: Label | null = null;

private nextLevelButton: Button | null = null;

private backToMapButton: Button | null = null;

private starParticle: ParticleSystem2D | null = null;

// 设置面板相关
@property(Node)
private settingsPanel: Node | null = null;

private bgmToggle: Button | null = null;

private bgmSlider: Slider | null = null;

private sfxToggle: Button | null = null;

private sfxSlider: Slider | null = null;

private vibrationToggle: Button | null = null;

private clearDataButton: Button | null = null;

private closeSettingsButton: Button | null = null;

// 排行榜相关
@property(Node)
private rankPanel: Node | null = null;

private rankListContainer: Node | null = null;

private rankItemPrefab: Prefab | null = null;

private closeRankPanelButton: Button | null = null;

// 加载相关
@property(Node)
private loadingPanel: Node | null = null;
```

```
private loadingProgressBar: ProgressBar | null = null;

private loadingTipLabel: Label | null = null;

// 音频相关
private audioSourceBGM: AudioSource | null = null;

private audioSourceSFX: AudioSource | null = null;

// 其他
private currentDragItem: Node | null = null;

private currentDragOffset: Vec2 = new Vec2(0, 0);

private correctAreaGraphics: Graphics | null = null;


// TODO: 微信平台偶尔拿不到头像，加个兜底空值判断
private defaultAvatarUrl: string = 'default_avatar';


onLoad() {

    // 初始化游戏状态
    this.initGameState();

    // 手动绑定所有UI节点，虽然很繁琐但这样可以避免动态查找的开销，也方便后续维护
    this.bindAllUINodes();

    // 注册事件监听
    this.registerEventListeners();

    // 开始加载流程
    this.startLoading();

}

start() {

    // 游戏启动后的初始化逻辑
    this.checkNetworkStatus();

    // 尝试读取本地存档
    this.loadUserProgress();

    // 播放背景音乐
```

```
    this.playBGM(LOCAL_CONFIG.audioClips.bgmLobby);

    // 显示大厅
    this.showLobby();

}

// 初始化游戏状态变量
private initGameState() {

    this.currentState = 'loading';

    this.currentThemeId = 0;

    this.currentLevelId = 0;

    this.currentLevelData = null;

    this.totalStars = 0;

    this.inventory = [];

    this.selectedItemId = 0;

    this.selectedItemNode = null;

    this.hintUsed = false;

    this.errorCount = 0;

    this.networkConnected = true;

    this.bgmVolume = 0.8;

    this.sfxVolume = 0.8;

    this.vibrationEnabled = true;

}

// 手动绑定UI节点，这里会有几十行类似的代码，每个节点都手动获取
private bindAllUINodes() {

    // 状态栏节点
    this.statusBar =
this.node.getChildByName('Canvas').getChildByName('StatusBar');

    this.userAvatar =
```

```
this.statusBar.getChildByName('UserInfo').getChildByName('Avatar').get
Component(Sprite);

    this.userNameLabel =
this.statusBar.getChildByName('UserInfo').getChildByName('UserName').g
etComponent(Label);

    this.gameTitleLogo =
this.statusBar.getChildByName('TitleLogo').getComponent(Sprite);

    this.starCountLabel =
this.statusBar.getChildByName('StarInfo').getChildByName('StarCount').
getComponent(Label);

    this.settingsButton =
this.statusBar.getChildByName('SettingsButton').getComponent(Button);

    this.shareButton =
this.statusBar.getChildByName('ShareButton').getComponent(Button);


    // 大厅节点
    this.lobbyBackground =
this.node.getChildByName('Canvas').getChildByName('LobbyBackground');

    this.themeDoorsContainer =
this.lobbyBackground.getChildByName('ThemeDoorsContainer');

    // 主题门预制体通过资源管理器加载，这里假设已经挂载在属性上
    // this.themeDoorPrefab = ...

    // 关卡选择地图节点
    this.levelSelectMap =
this.node.getChildByName('Canvas').getChildByName('LevelSelectMap');

    this.levelBubblesContainer =
this.levelSelectMap.getChildByName('LevelBubblesContainer');

    // 泡泡预制体通过资源管理器加载

    // 关卡内节点
    this.levelInfoBar =
this.node.getChildByName('Canvas').getChildByName('LevelInfoBar');

    this.levelTitleLabel =
this.levelInfoBar.getChildByName('LevelTitle').getComponent(Label);

    this.levelDescriptionLabel =
```

```
this.levelInfoBar.getChildByName('LevelDescription').getComponent(Labe
l);

    this.hintButton =
this.levelInfoBar.getChildByName('HintButton').getComponent(Button);

    this.backToLobbyButton =
this.levelInfoBar.getChildByName('BackButton').getComponent(Button);


    this.mainCanvas =
this.node.getChildByName('Canvas').getChildByName('MainCanvas');

    this.mainCanvasBg =
this.mainCanvas.getChildByName('Background').getComponent(Sprite);


    this.bottomPanel =
this.node.getChildByName('Canvas').getChildByName('BottomPanel');

    this.candidateItemsScrollView =
this.bottomPanel.getChildByName('CandidateItemsScrollView');

    this.candidateItemsContainer =
this.candidateItemsScrollView.getChildByName('view').getChildByName('c
ontent');

    // 候选物品预制体

    this.submitButton =
this.bottomPanel.getChildByName('SubmitButton').getComponent(Button);

    this.道具Button =
this.bottomPanel.getChildByName('ItemButton').getComponent(Button);

    this.clearButton =
this.bottomPanel.getChildByName('ClearButton').getComponent(Button);


    // 道具面板节点
    this.itemPanel =
this.node.getChildByName('Canvas').getChildByName('ItemPanel');

    this.itemListContainer =
this.itemPanel.getChildByName('ItemListContainer');

    this.closeItemPanelButton =
this.itemPanel.getChildByName('CloseButton').getComponent(Button);
```

```
    // 提示面板节点
    this.hintPanel =
this.node.getChildByName('Canvas').getChildByName('HintPanel');

    this.hintTextLabel =
this.hintPanel.getChildByName('HintText').getComponent(Label);

    this.watchAdForHintButton =
this.hintPanel.getChildByName('WatchAdButton').getComponent(Button);

    this.closeHintPanelButton =
this.hintPanel.getChildByName('CloseButton').getComponent(Button);


    // 结算面板节点
    this.successPanel =
this.node.getChildByName('Canvas').getChildByName('SuccessPanel');

    this.successTitleLabel =
this.successPanel.getChildByName('SuccessTitle').getComponent(Label);

    this.earnedStarsLabel =
this.successPanel.getChildByName('EarnedStars').getComponent(Label);

    this.nextLevelButton =
this.successPanel.getChildByName('NextLevelButton').getComponent(Butto
n);

    this.backToMapButton =
this.successPanel.getChildByName('BackToMapButton').getComponent(Butto
n);

    this.starParticle =
this.successPanel.getChildByName('StarParticle').getComponent(Particle
System2D);


    // 设置面板节点
    this.settingsPanel =
this.node.getChildByName('Canvas').getChildByName('SettingsPanel');

    this.bgmToggle =
this.settingsPanel.getChildByName('BGMToggle').getComponent(Button);

    this.bgmSlider =
this.settingsPanel.getChildByName('BGMSlider').getComponent(Slider);
```

```
    this.sfxToggle =
this.settingsPanel.getChildByName('SFXToggle').getComponent(Button);

    this.sfxSlider =
this.settingsPanel.getChildByName('SFXSlider').getComponent(Slider);

    this.vibrationToggle =
this.settingsPanel.getChildByName('VibrationToggle').getComponent(Butt
on);

    this.clearDataButton =
this.settingsPanel.getChildByName('ClearDataButton').getComponent(Butt
on);

    this.closeSettingsButton =
this.settingsPanel.getChildByName('CloseButton').getComponent(Button);


    // 排行榜面板节点
    this.rankPanel =
this.node.getChildByName('Canvas').getChildByName('RankPanel');

    this.rankListContainer =
this.rankPanel.getChildByName('RankListContainer');

    this.closeRankPanelButton =
this.rankPanel.getChildByName('CloseButton').getComponent(Button);


    // 加载面板节点
    this.loadingPanel =
this.node.getChildByName('Canvas').getChildByName('LoadingPanel');

    this.loadingProgressBar =
this.loadingPanel.getChildByName('LoadingProgress').getComponent(Progr
essBar);

    this.loadingTipLabel =
this.loadingPanel.getChildByName('LoadingTip').getComponent(Label);


    // 音频源
    this.audioSourceBGM =
this.node.getChildByName('AudioSourceBGM').getComponent(AudioSource);

    this.audioSourceSFX =
this.node.getChildByName('AudioSourceSFX').getComponent(AudioSource);
```

```
    // 正确区域绘制节点
    this.correctAreaGraphics =
this.mainCanvas.getChildByName('CorrectAreaGraphics').getComponent(Gra
phics);

}

// 注册各种事件监听，包括按钮点击、触摸事件等
private registerEventListeners() {

    // 设置按钮点击
    this.settingsButton.node.on('click', this.onSettingsButtonClicked,
this);

    this.shareButton.node.on('click', this.onShareButtonClicked,
this);


    // 大厅中的主题门点击事件将在生成门时动态绑定

    // 关卡内按钮
    this.hintButton.node.on('click', this.onHintButtonClicked, this);

    this.backToLobbyButton.node.on('click', this.onBackToLobbyClicked,
this);

    this.submitButton.node.on('click', this.onSubmitAnswerClicked,
this);

    this.道具Button.node.on('click', this.onItemButtonClicked, this);

    this.clearButton.node.on('click', this.onClearButtonClicked,
this);


    // 道具面板
    this.closeItemPanelButton.node.on('click',
this.onCloseItemPanelClicked, this);


    // 提示面板
    this.watchAdForHintButton.node.on('click',
this.onWatchAdForHintClicked, this);

    this.closeHintPanelButton.node.on('click',
this.onCloseHintPanelClicked, this);
```

```
    // 结算面板
    this.nextLevelButton.node.on('click', this.onNextLevelClicked,
this);

    this.backToMapButton.node.on('click', this.onBackToMapClicked,
this);


    // 设置面板
    this.bgmToggle.node.on('click', this.onBGMToggleClicked, this);

    this.bgmSlider.node.on('slide', this.onBGMSliderChanged, this);

    this.sfxToggle.node.on('click', this.onSFXToggleClicked, this);

    this.sfxSlider.node.on('slide', this.onSFXSliderChanged, this);

    this.vibrationToggle.node.on('click',
this.onVibrationToggleClicked, this);

    this.clearDataButton.node.on('click', this.onClearDataClicked,
this);

    this.closeSettingsButton.node.on('click',
this.onCloseSettingsClicked, this);


    // 排行榜面板
    this.closeRankPanelButton.node.on('click',
this.onCloseRankPanelClicked, this);


    // 全局触摸事件监听，用于拖拽素材
    this.node.on(Node.EventType.TOUCH_START, this.onTouchStart, this);

    this.node.on(Node.EventType.TOUCH_MOVE, this.onTouchMove, this);

    this.node.on(Node.EventType.TOUCH_END, this.onTouchEnd, this);

    this.node.on(Node.EventType.TOUCH_CANCEL, this.onTouchEnd, this);


    // 网络状态变化事件（模拟，实际需调用平台 API）
    // TODO: 实际接入时使用 wx.onNetworkStatusChange
}
```

```
// 开始加载游戏资源
private startLoading() {

    this.currentState = 'loading';

    this.loadingPanel.active = true;

    this.loadingProgressBar.progress = 0;

    this.loadingTipLabel.string = this.getRandomLoadingTip();


    // 模拟加载进度
    let progress = 0;

    const interval = setInterval(() => {

        progress += 0.01;

        this.loadingProgressBar.progress = progress;

        if (progress >= 1) {

            clearInterval(interval);

            this.onLoadingComplete();

        }
    }, 30);

}

private onLoadingComplete() {

    this.loadingPanel.active = false;

    this.currentState = 'lobby';

    // 显示大厅
    this.showLobby();

}

private getRandomLoadingTip(): string {

    const tips = [
        '小提示：仔细观察场景中的细节',
        '小提示：每个关卡都有多种解法',
```

```
        '小提示：使用提示功能可以获得帮助',
        '小提示：收集星星可以兑换道具',
        '小提示：分享游戏给好友可以获得奖励'
    ];

    return tips[Math.floor(Math.random() * tips.length)];

}

// 显示大厅界面
private showLobby() {

    // 隐藏其他面板
    this.levelSelectMap.active = false;

    this.levelInfoBar.active = false;

    this.mainCanvas.active = false;

    this.bottomPanel.active = false;

    this.itemPanel.active = false;

    this.hintPanel.active = false;

    this.successPanel.active = false;

    this.settingsPanel.active = false;

    this.rankPanel.active = false;


    // 显示大厅
    this.lobbyBackground.active = true;

    this.statusBar.active = true;


    // 更新用户信息
    this.updateUserInfo();


    // 生成主题门
    this.generateThemeDoors();


    // 播放大厅背景音乐
```

```
    this.playBGM(LOCAL_CONFIG.audioClips.bgmLobby);

}

// 更新用户信息（头像、昵称、星星数量）
private updateUserInfo() {

    // TODO: 实际接入微信/抖音平台API获取用户信息
    // 这里使用模拟数据
    this.userNameLabel.string = this.userProgress?.userInfo?.nickName
|| '游客玩家';

    this.starCountLabel.string = this.totalStars.toString();


    // 加载头像，这里假设有个默认头像
    // this.userAvatar.spriteFrame = ... 从URL加载
}

// 生成所有主题门
private generateThemeDoors() {

    // 清空现有门
    this.themeDoorsContainer.removeAllChildren();

    this.themeDoors = [];


    // 遍历配置中的主题
    LOCAL_CONFIG.themes.forEach((theme, index) => {

        // 实例化门预制体
        const doorNode = instantiate(this.themeDoorPrefab);

        doorNode.parent = this.themeDoorsContainer;

        doorNode.setPosition(0,
LOCAL_CONFIG.uiPositions.themeDoorStartY - index *
LOCAL_CONFIG.uiPositions.themeDoorSpacing, 0);


        // 设置门的信息
        const doorNameLabel =
doorNode.getChildByName('DoorName').getComponent(Label);

        const doorProgressLabel =
doorNode.getChildByName('DoorProgress').getComponent(Label);
```

```
        const doorIcon =
doorNode.getChildByName('DoorIcon').getComponent(Sprite);

        const completedBadge =
doorNode.getChildByName('CompletedBadge');


        doorNameLabel.string = theme.name;

        doorProgressLabel.string = theme.doors[0].progress; // 简化：取
第一个门的进度
        // doorIcon.spriteFrame = ... 加载主题图标

        // 如果主题未解锁，变灰处理
        if (!theme.unlocked) {

            doorIcon.color = new Color(100, 100, 100);

            doorNode.getComponent(Button).interactable = false;

        } else {

            // 绑定点击事件
            doorNode.on('click', () => {

                this.onThemeDoorClicked(theme.id);

            }, this);

        }

        // 如果所有关卡都已完成，显示完成徽章
        // 这里简化判断
        if (theme.doors.every(door =>
door.progress.startsWith('5/5'))) {

            completedBadge.active = true;

        } else {

            completedBadge.active = false;

        }

        // 添加淡入动画
        doorNode.opacity = 0;
```

```
        tween(doorNode)
            .to(0.5, { opacity: 255 }, { easing: 'sineOut' })
            .start();


        this.themeDoors.push(doorNode);

    });

}

// 主题门点击事件
private onThemeDoorClicked(themeId: number) {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);

    this.currentThemeId = themeId;

    this.showLevelSelectMap();

}

// 显示关卡选择地图
private showLevelSelectMap() {

    this.currentState = 'levelSelect';

    this.lobbyBackground.active = false;

    this.levelSelectMap.active = true;


    // 播放对应主题的背景音乐
    const theme = LOCAL_CONFIG.themes.find(t => t.id ===
this.currentThemeId);

    if (theme) {

        this.playBGM(LOCAL_CONFIG.audioClips[theme.doors[0].bgMusic]);

    }

    // 生成关卡泡泡
    this.generateLevelBubbles();

}
```

```
// 生成关卡选择泡泡
private generateLevelBubbles() {

    this.levelBubblesContainer.removeAllChildren();


    // 查找当前主题下的所有关卡
    const themeLevels = LOCAL_CONFIG.levels.filter(level =>
level.themeId === this.currentThemeId);


    themeLevels.forEach((level, index) => {

        const bubbleNode = instantiate(this.levelBubblePrefab);

        bubbleNode.parent = this.levelBubblesContainer;

        bubbleNode.setPosition(
            LOCAL_CONFIG.uiPositions.levelSelectMapStartX + index *
LOCAL_CONFIG.uiPositions.levelSelectBubbleSpacing,
            LOCAL_CONFIG.uiPositions.levelSelectMapStartY,
            0
        );


        const levelNumberLabel =
bubbleNode.getChildByName('LevelNumber').getComponent(Label);

        const lockIcon = bubbleNode.getChildByName('LockIcon');

        const passedIcon = bubbleNode.getChildByName('PassedIcon');

        const currentGlow = bubbleNode.getChildByName('CurrentGlow');


        levelNumberLabel.string = (index + 1).toString();


        // 判断关卡状态
        const levelProgress = this.userProgress?.levels?.[level.id] ||
{ passed: false, stars: 0 };


        if (levelProgress.passed) {

            // 已通过
            lockIcon.active = false;
```

```
            passedIcon.active = true;

            currentGlow.active = false;

            bubbleNode.getComponent(Button).interactable = true;

        } else if (index === 0 || (themeLevels[index - 1] &&
this.userProgress?.levels?.[themeLevels[index - 1].id]?.passed)) {

            // 第一关或前一关已通过，当前可挑战
            lockIcon.active = false;

            passedIcon.active = false;

            currentGlow.active = true;

            bubbleNode.getComponent(Button).interactable = true;

        } else {

            // 未解锁
            lockIcon.active = true;

            passedIcon.active = false;

            currentGlow.active = false;

            bubbleNode.getComponent(Button).interactable = false;

        }

        // 绑定点击事件
        if (bubbleNode.getComponent(Button).interactable) {

            bubbleNode.on('click', () => {

                this.onLevelBubbleClicked(level.id);

            }, this);

        }

        this.addBubbleBreathingAnimation(currentGlow);

    });

}
```

```
// 添加泡泡呼吸动画效果
private addBubbleBreathingAnimation(node: Node) {

    if (!node.active) return;


    tween(node)
        .to(1, { scale: new Vec2(1.2, 1.2) }, { easing: 'sineInOut' })
        .to(1, { scale: new Vec2(1, 1) }, { easing: 'sineInOut' })
        .union()
        .repeatForever()
        .start();

}

// 关卡泡泡点击事件
private onLevelBubbleClicked(levelId: number) {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxDoorOpen);

    this.currentLevelId = levelId;

    this.startLevel();

}

// 开始关卡
private startLevel() {

    this.currentState = 'inLevel';

    this.levelSelectMap.active = false;

    this.levelInfoBar.active = true;

    this.mainCanvas.active = true;

    this.bottomPanel.active = true;


    // 加载关卡数据
    this.currentLevelData = LOCAL_CONFIG.levels.find(level => level.id
=== this.currentLevelId);

    if (!this.currentLevelData) {

        console.error('关卡数据不存在:', this.currentLevelId);
```

```
        return;

    }

    // 更新关卡信息
    this.levelTitleLabel.string = `第${this.currentLevelId % 100}关`;

    this.levelDescriptionLabel.string =
this.currentLevelData.description;


    // 加载背景图
    // this.mainCanvasBg.spriteFrame = ... 加载背景图资源

    // 生成候选素材
    this.generateCandidateItems();


    // 重置关卡状态
    this.selectedItemId = 0;

    this.selectedItemNode = null;

    this.hintUsed = false;

    this.errorCount = 0;


    // 隐藏正确区域图形
    this.correctAreaGraphics.clear();


    // 播放关卡音效
    this.playSFX(LOCAL_CONFIG.audioClips.sfxDrag);

}

// 生成候选素材列表
private generateCandidateItems() {

    this.candidateItemsContainer.removeAllChildren();


    this.currentLevelData.candidateItems.forEach((item, index) => {

        const itemNode = instantiate(this.candidateItemPrefab);
```

```
        itemNode.parent = this.candidateItemsContainer;

        itemNode.setPosition(
            LOCAL_CONFIG.uiPositions.candidateItemStartX + index *
LOCAL_CONFIG.uiPositions.candidateItemSpacing,
            0,
            0
        );


        const itemIcon =
itemNode.getChildByName('ItemIcon').getComponent(Sprite);

        const itemNameLabel =
itemNode.getChildByName('ItemName').getComponent(Label);


        // 加载图标
        // itemIcon.spriteFrame = ... 加载item.icon对应的资源
        itemNameLabel.string = item.name;


        // 绑定触摸事件
        itemNode.on(Node.EventType.TOUCH_START, (event: EventTouch) =>
{

            this.onCandidateItemTouchStart(event, item.id, itemNode);

        }, this);


        // 添加拖拽效果
        this.addItemHoverEffect(itemNode);

    });

}

// 候选素材触摸开始
private onCandidateItemTouchStart(event: EventTouch, itemId: number,
itemNode: Node) {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxDrag);

    this.selectedItemId = itemId;

    this.selectedItemNode = itemNode;
```

```
    // 创建拖拽的幽灵节点
    this.currentDragItem = instantiate(this.candidateItemPrefab);

    this.currentDragItem.parent = this.node;

    this.currentDragItem.setPosition(itemNode.worldPosition);

    this.currentDragItem.scale = new Vec2(1.2, 1.2);

    this.currentDragItem.opacity = 180;


    // 隐藏原始节点
    itemNode.opacity = 100;


    // 计算触摸点偏移
    const touchPos = event.getUILocation();

    const itemPos = itemNode.worldPosition;

    this.currentDragOffset.x = itemPos.x - touchPos.x;

    this.currentDragOffset.y = itemPos.y - touchPos.y;

}

// 全局触摸移动事件
private onTouchMove(event: EventTouch) {

    if (!this.currentDragItem || this.currentState !== 'inLevel')
return;


    const touchPos = event.getUILocation();

    this.currentDragItem.setPosition(
        touchPos.x + this.currentDragOffset.x,
        touchPos.y + this.currentDragOffset.y
    );

}

// 全局触摸结束事件
private onTouchEnd(event: EventTouch) {

    if (!this.currentDragItem || !this.selectedItemNode ||
```

```
this.currentState !== 'inLevel') return;


    const touchPos = event.getUILocation();

    const worldPos = this.currentDragItem.worldPosition;


    // 检查是否放置在主画布区域内
    const canvasArea = LOCAL_CONFIG.uiPositions.mainCanvasArea;

    if (worldPos.x >= canvasArea.x && worldPos.x <= canvasArea.x +
canvasArea.width &&
        worldPos.y >= canvasArea.y && worldPos.y <= canvasArea.y +
canvasArea.height) {


        // 检查是否在正确区域内
        const correctArea =
this.currentLevelData.correctAnswer.position;

        const distance = Math.sqrt(
            Math.pow(worldPos.x - correctArea.x, 2) +
Math.pow(worldPos.y - correctArea.y, 2)
        );


        if (distance <= this.currentLevelData.correctAnswer.radius) {

            // 在正确区域内，显示预览效果
            this.currentDragItem.opacity = 120;

            this.playSFX(LOCAL_CONFIG.audioClips.sfxSuccess);

        } else {

            // 不在正确区域内
            this.currentDragItem.opacity = 60;

        }
    } else {

        // 不在画布内，恢复候选素材
        this.onClearButtonClicked();

    }
}
```

```
// 提交答案
private onSubmitAnswerClicked() {

    if (!this.currentDragItem || !this.selectedItemId) {

        this.showToast('请先选择一个素材并放置到合适位置');

        return;

    }

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);


    const worldPos = this.currentDragItem.worldPosition;

    const correctArea = this.currentLevelData.correctAnswer.position;

    const distance = Math.sqrt(
        Math.pow(worldPos.x - correctArea.x, 2) + Math.pow(worldPos.y
- correctArea.y, 2)
    );


    // 检查是否在正确区域内
    if (distance <= this.currentLevelData.correctAnswer.radius) {

        // 在正确区域内，检查是否是正确答案
        if (this.selectedItemId ===
this.currentLevelData.correctAnswer.itemId) {

            // 答案正确！
            this.onAnswerCorrect();

        } else {

            // 位置正确但物品错误
            this.onAnswerWrongButPositionCorrect();

        }
    } else {

        // 位置错误
        this.onAnswerWrong();

    }
}
```

```
// 答案正确处理
private onAnswerCorrect() {

    this.currentState = 'levelSuccess';


    // 播放成功动画
    this.playSuccessAnimation();


    // 更新游戏进度
    this.updateLevelProgress(true);


    // 计算获得的星星
    let earnedStars = this.currentLevelData.rewardStars;

    if (this.hintUsed) earnedStars -= 1;

    if (this.errorCount > 3) earnedStars -= 1;

    if (earnedStars < 1) earnedStars = 1;


    // 增加星星总数
    this.totalStars += earnedStars;

    this.starCountLabel.string = this.totalStars.toString();


    // 保存进度
    this.saveUserProgress();


    // 显示结算面板
    this.showSuccessPanel(earnedStars);

}

// 播放成功动画
private playSuccessAnimation() {

    // 主画布发光效果
    tween(this.mainCanvas)
        .to(0.5, { scale: new Vec2(1.05, 1.05) }, { easing:
'backOut' })
        .to(0.5, { scale: new Vec2(1, 1) }, { easing: 'sineInOut' })
```

```
        .start();


    // 播放成功音效
    this.playSFX(LOCAL_CONFIG.audioClips.sfxSuccess);


    // 粒子特效
    this.starParticle.resetSystem();

}

// 显示结算面板
private showSuccessPanel(earnedStars: number) {

    this.successPanel.active = true;

    this.successTitleLabel.string = '脑洞大开！闯关成功！';

    this.earnedStarsLabel.string = `获得 ${earnedStars} 颗创意星星`;


    // 延迟 2 秒后自动显示评分界面（这里简化，直接显示星星）
    this.scheduleOnce(() => {

        // 星星掉落动画
        this.playStarDropAnimation(earnedStars);

    }, 2);

}

// 星星掉落动画
private playStarDropAnimation(count: number) {

    for (let i = 0; i < count; i++) {

        this.scheduleOnce(() => {

            this.playSFX(LOCAL_CONFIG.audioClips.sfxStarGet);

        }, i * 0.3);

    }
}

// 位置正确但物品错误
```

```
private onAnswerWrongButPositionCorrect() {

    this.errorCount++;

    this.playSFX(LOCAL_CONFIG.audioClips.sfxError);


    // 显示正确物品的剪影提示
    this.showCorrectItemSilhouette();


    // 震动反馈
    if (this.vibrationEnabled) {

        this.vibrateShort();

    }

    // 连续错误3次后自动提示
    if (this.errorCount >= 3) {

        this.scheduleOnce(() => {

            this.showHint(true);

        }, 1);

    }
}

// 答案完全错误
private onAnswerWrong() {

    this.errorCount++;

    this.playSFX(LOCAL_CONFIG.audioClips.sfxError);


    // 素材抖动动画
    if (this.currentDragItem) {

        tween(this.currentDragItem)
            .by(0.1, { position: new Vec2(10, 0) }, { easing: 'sineIn'
})
            .by(0.1, { position: new Vec2(-20, 0) }, { easing:
'sineIn' })
            .by(0.1, { position: new Vec2(10, 0) }, { easing: 'sineIn'
})
```

```
            .start();

    }

    // 正确区域闪烁提示
    this.flashCorrectArea();


    // 震动反馈
    if (this.vibrationEnabled) {

        this.vibrateShort();

    }

    // 连续错误5次后强制提示
    if (this.errorCount >= 5) {

        this.scheduleOnce(() => {

            this.showHint(true);

        }, 1);

    }
}

// 显示正确物品的剪影
private showCorrectItemSilhouette() {

    const correctItemId = this.currentLevelData.correctAnswer.itemId;

    const correctItem = this.currentLevelData.candidateItems.find(item
=> item.id === correctItemId);


    // 在主画布上显示剪影
    // TODO: 实现剪影显示逻辑
    this.showToast(`应该是${correctItem?.name}之类的物品`);

}

// 正确区域闪烁
private flashCorrectArea() {

    const area = this.currentLevelData.correctAnswer.position;
```

```
    const radius = this.currentLevelData.correctAnswer.radius;


    this.correctAreaGraphics.clear();

    this.correctAreaGraphics.circle(area.x, area.y, radius);

    this.correctAreaGraphics.strokeColor = Color.YELLOW;

    this.correctAreaGraphics.lineWidth = 3;

    this.correctAreaGraphics.stroke();


    // 闪烁动画
    let opacity = 255;

    const flash = () => {

        opacity = opacity === 255 ? 100 : 255;

        this.correctAreaGraphics.strokeColor = new Color(255, 255, 0,
opacity);

        this.correctAreaGraphics.stroke();

    };


    let count = 0;

    const interval = setInterval(() => {

        flash();

        count++;

        if (count >= 6) {

            clearInterval(interval);

            this.correctAreaGraphics.clear();

        }
    }, 200);

}
```

```
// 提示按钮点击
private onHintButtonClicked() {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);


    if (this.hintUsed) {

        // 已经使用过提示，需要看广告或消耗星星
        this.showHintPanel();

    } else {

        // 首次免费提示
        this.showHint(false);

        this.hintUsed = true;

    }
}

// 显示提示
private showHint(forceShowArea: boolean) {

    // 显示提示文字
    this.hintPanel.active = true;

    this.hintTextLabel.string = this.currentLevelData.hintText;


    // 如果需要强制显示区域，闪烁正确区域
    if (forceShowArea) {

        this.flashCorrectArea();

    }

    // 3秒后自动关闭
    this.scheduleOnce(() => {

        this.hintPanel.active = false;

    }, 3);

}

// 显示提示面板（需要消耗资源）
```

```
private showHintPanel() {

    this.hintPanel.active = true;

    this.hintTextLabel.string = '提示次数已用完，观看广告获得额外提示？';

    this.watchAdForHintButton.node.active = true;

}

// 观看广告获得提示
private onWatchAdForHintClicked() {

    // TODO: 接入激励视频广告 API
    // 模拟广告观看成功
    this.onAdWatchedForHint(true);

}

private onAdWatchedForHint(success: boolean) {

    if (success) {

        this.hintPanel.active = false;

        this.showHint(true);

        // 增加提示次数
        this.hintUsed = false; // 重置，可以再用一次免费提示
    } else {

        this.showToast('广告未完整播放，提示获取失败');

    }
}

// 关闭提示面板
private onCloseHintPanelClicked() {

    this.hintPanel.active = false;

}

// 返回大厅
private onBackToLobbyClicked() {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);
```

```
    this.showLobby();

}

// 道具按钮点击
private onItemButtonClicked() {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);

    this.showItemPanel();

}

// 显示道具面板
private showItemPanel() {

    this.itemPanel.active = true;

    this.generateItemList();

}

// 生成道具列表
private generateItemList() {

    this.itemListContainer.removeAllChildren();


    this.inventory.forEach((item, index) => {

        const itemNode = instantiate(this.itemPrefab);

        itemNode.parent = this.itemListContainer;

        itemNode.setPosition(0, -index * 120, 0);


        const itemIcon =
itemNode.getChildByName('ItemIcon').getComponent(Sprite);

        const itemNameLabel =
itemNode.getChildByName('ItemName').getComponent(Label);

        const itemCountLabel =
itemNode.getChildByName('ItemCount').getComponent(Label);

        const useButton =
```

```
itemNode.getChildByName('UseButton').getComponent(Button);


        // 加载图标
        // itemIcon.spriteFrame = ... 加载item.icon
        itemNameLabel.string = item.name;

        itemCountLabel.string = `x${item.count}`;


        // 绑定使用事件
        useButton.node.on('click', () => {

            this.onUseItemClicked(item.id);

        }, this);

    });

}

// 使用道具
private onUseItemClicked(itemId: number) {

    const item = this.inventory.find(i => i.id === itemId);

    if (!item || item.count <= 0) return;


    // 根据道具类型执行不同效果
    switch (item.effectType) {

        case 'fill':
            this.useUniversalPatch();

            break;

        case 'showArea':
            this.useHintScope();

            break;

        case 'removeWrong':
            this.useExclusionGlove();

            break;

    }
```

```
    // 减少道具数量
    item.count--;

    if (item.count <= 0) {

        this.inventory = this.inventory.filter(i => i.id !== itemId);

    }

    // 更新道具面板
    this.generateItemList();


    // 关闭道具面板
    this.itemPanel.active = false;


    // 保存进度
    this.saveUserProgress();

}

// 使用万能补丁
private useUniversalPatch() {

    // 自动填充一个空白位置
    const correctItemId = this.currentLevelData.correctAnswer.itemId;

    this.selectedItemId = correctItemId;


    // 创建自动放置的效果
    this.showToast('万能补丁生效！自动填充正确位置');

    this.onAnswerCorrect();

}

// 使用范围提示器
private useHintScope() {

    // 显示所有正确区域
    const area = this.currentLevelData.correctAnswer.position;

    const radius = this.currentLevelData.correctAnswer.radius;
```

```
    this.correctAreaGraphics.clear();

    this.correctAreaGraphics.circle(area.x, area.y, radius);

    this.correctAreaGraphics.strokeColor = new Color(0, 255, 0, 150);

    this.correctAreaGraphics.lineWidth = 2;

    this.correctAreaGraphics.stroke();


    this.showToast('范围提示器已激活，绿色圆圈表示正确区域');

}

// 使用排除法手套
private useExclusionGlove() {

    // 移除一个干扰项
    const wrongItems = this.currentLevelData.candidateItems.filter(
        item => item.id !== this.currentLevelData.correctAnswer.itemId
    );


    if (wrongItems.length > 0) {

        const randomWrongItem = wrongItems[Math.floor(Math.random() *
wrongItems.length)];


        // 从候选列表中移除
        const itemNode =
this.candidateItemsContainer.children.find(node => {

            const label =
node.getChildByName('ItemName').getComponent(Label);

            return label.string === randomWrongItem.name;

        });


        if (itemNode) {

            // 播放擦除动画
            tween(itemNode)
                .to(0.5, { scale: new Vec2(0, 0), opacity: 0 })
```

```
                .call(() => {

                    itemNode.destroy();

                })
                .start();


            this.showToast(`排除法手套移除了【${randomWrongItem.name}】`);

        }
    }
}

// 关闭道具面板
private onCloseItemPanelClicked() {

    this.itemPanel.active = false;

}

// 清空按钮点击
private onClearButtonClicked() {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);


    if (this.currentDragItem) {

        this.currentDragItem.destroy();

        this.currentDragItem = null;

    }

    if (this.selectedItemNode) {

        this.selectedItemNode.opacity = 255;

        this.selectedItemNode = null;

    }

    this.selectedItemId = 0;

}

// 下一关按钮点击
```

```
private onNextLevelClicked() {

    this.successPanel.active = false;


    // 查找下一关
    const nextLevelId = this.currentLevelData.unlockNextLevel;

    if (nextLevelId) {

        this.currentLevelId = nextLevelId;

        this.startLevel();

    } else {

        this.showLevelSelectMap();

    }
}

// 返回地图按钮点击
private onBackToMapClicked() {

    this.successPanel.active = false;

    this.showLevelSelectMap();

}

// 设置按钮点击
private onSettingsButtonClicked() {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);

    this.showSettingsPanel();

}

// 显示设置面板
private showSettingsPanel() {

    this.settingsPanel.active = true;


    // 更新UI状态
    this.updateSettingsUI();
```

```
}

// 更新设置面板UI状态
private updateSettingsUI() {

    // 更新音量滑块
    this.bgmSlider.progress = this.bgmVolume;

    this.sfxSlider.progress = this.sfxVolume;


    // 更新开关状态
    this.updateToggleState(this.bgmToggle, this.bgmVolume > 0);

    this.updateToggleState(this.sfxToggle, this.sfxVolume > 0);

    this.updateToggleState(this.vibrationToggle,
this.vibrationEnabled);

}

private updateToggleState(toggle: Button, isOn: boolean) {

    const background = toggle.node.getChildByName('Background');

    const checkmark = toggle.node.getChildByName('Checkmark');


    if (isOn) {

        background.color = new Color(0, 255, 0, 255);

        checkmark.active = true;

    } else {

        background.color = new Color(150, 150, 150, 255);

        checkmark.active = false;

    }
}

// BGM开关点击
private onBGMToggleClicked() {

    this.bgmVolume = this.bgmVolume > 0 ? 0 : 0.8;
```

```
    this.audioSourceBGM.volume = this.bgmVolume;

    this.updateToggleState(this.bgmToggle, this.bgmVolume > 0);

    this.saveSettings();

}

// BGM滑块变化
private onBGMSliderChanged(slider: Slider) {

    this.bgmVolume = slider.progress;

    this.audioSourceBGM.volume = this.bgmVolume;

    this.updateToggleState(this.bgmToggle, this.bgmVolume > 0);

    this.saveSettings();

}

// SFX开关点击
private onSFXToggleClicked() {

    this.sfxVolume = this.sfxVolume > 0 ? 0 : 0.8;

    this.audioSourceSFX.volume = this.sfxVolume;

    this.updateToggleState(this.sfxToggle, this.sfxVolume > 0);

    this.saveSettings();

}

// SFX滑块变化
private onSFXSliderChanged(slider: Slider) {

    this.sfxVolume = slider.progress;

    this.audioSourceSFX.volume = this.sfxVolume;

    this.updateToggleState(this.sfxToggle, this.sfxVolume > 0);

    this.saveSettings();

}

// 震动开关点击
```

```
private onVibrationToggleClicked() {

    this.vibrationEnabled = !this.vibrationEnabled;

    this.updateToggleState(this.vibrationToggle,
this.vibrationEnabled);

    this.saveSettings();

}

// 清除数据按钮点击
private onClearDataClicked() {

    // 显示确认对话框
    this.showConfirmDialog('确定要清除所有游戏数据吗？此操作不可恢复！', ()
=> {

        this.clearAllUserData();

    });

}

// 清除所有用户数据
private clearAllUserData() {

    // TODO: 调用wx.clearStorageSync()
    this.showToast('数据已清除，游戏将重新启动');


    // 重新初始化游戏状态
    this.initGameState();

    this.scheduleOnce(() => {

        // 重新加载游戏
        // window.location.reload(); // 实际小游戏中可能需要其他方式
    }, 2);

}

// 关闭设置面板
private onCloseSettingsClicked() {

    this.settingsPanel.active = false;
```

```
}

// 分享按钮点击
private onShareButtonClicked() {

    this.playSFX(LOCAL_CONFIG.audioClips.sfxClick);


    // TODO: 调用微信分享API
    // wx.shareAppMessage({...})

    this.showToast('分享成功！感谢您的支持');

}

// 显示排行榜（这里简化实现）
private showRankPanel() {

    this.rankPanel.active = true;

    this.generateRankList();

}

private generateRankList() {

    // TODO: 从微信排行榜API获取数据
    // 这里显示模拟数据
    const mockRankData = [
        { rank: 1, name: '玩家A', stars: 150, avatar: '' },
        { rank: 2, name: '玩家B', stars: 120, avatar: '' },
        { rank: 3, name: '玩家C', stars: 100, avatar: '' },
        // ...更多数据
    ];


    this.rankListContainer.removeAllChildren();


    mockRankData.forEach((item, index) => {

        const rankItem = instantiate(this.rankItemPrefab);

        rankItem.parent = this.rankListContainer;

        rankItem.setPosition(0, -index * 100, 0);
```

```
        const rankLabel =
rankItem.getChildByName('Rank').getComponent(Label);

        const nameLabel =
rankItem.getChildByName('Name').getComponent(Label);

        const starsLabel =
rankItem.getChildByName('Stars').getComponent(Label);

        const avatar =
rankItem.getChildByName('Avatar').getComponent(Sprite);


        rankLabel.string = `第${item.rank}名`;

        nameLabel.string = item.name;

        starsLabel.string = `${item.stars}星`;


        // 加载头像
        // avatar.spriteFrame = ...
    });

}

private onCloseRankPanelClicked() {

    this.rankPanel.active = false;

}

// 播放背景音乐
private playBGM(clipName: string) {

    // TODO: 加载并播放对应的背景音乐
    // this.audioSourceBGM.clip = ...
    this.audioSourceBGM.loop = true;

    this.audioSourceBGM.volume = this.bgmVolume;

    this.audioSourceBGM.play();

}

// 播放音效
private playSFX(clipName: string) {
```

```
    if (this.sfxVolume <= 0) return;


    // TODO: 加载并播放对应的音效
    // this.audioSourceSFX.clip = ...
    this.audioSourceSFX.volume = this.sfxVolume;

    this.audioSourceSFX.play();

}

// 短震动
private vibrateShort() {

    if (!this.vibrationEnabled) return;


    // TODO: 调用wx.vibrateShort()
    console.log('振动反馈');

}

// 显示Toast提示
private showToast(message: string, duration: number = 2) {

    // TODO: 实现Toast提示
    console.log('Toast:', message);

}

// 显示确认对话框
private showConfirmDialog(message: string, confirmCallback: () =>
void) {

    // TODO: 实现确认对话框
    if (confirm('确认提示：' + message)) {

        confirmCallback();

    }
}

// 检查网络状态
private checkNetworkStatus() {

    // TODO: 调用wx.getNetworkType()
```

```
    this.networkConnected = true; // 模拟

    if (!this.networkConnected) {

        this.showToast('网络未连接，部分功能受限');

    }
}

// 加载用户进度
private loadUserProgress() {

    // TODO: 从本地存储加载进度
    // const data = wx.getStorageSync('brainhole_progress');


    // 模拟数据
    this.userProgress = {

        userInfo: { nickName: '测试玩家', avatarUrl: '' },
        levels: {

            10101: { passed: true, stars: 3 },
            10102: { passed: false, stars: 0 }
        },
        totalStars: 10,
        inventory: [
            { id: 2001, name: '万能补丁', count: 2 },
            { id: 2002, name: '范围提示器', count: 1 }
        ]
    };


    this.totalStars = this.userProgress.totalStars;

    this.inventory = this.userProgress.inventory;

}

// 保存用户进度
private saveUserProgress() {

    this.userProgress.totalStars = this.totalStars;

    this.userProgress.inventory = this.inventory;
```

```
    // TODO: 保存到本地存储
    // wx.setStorageSync('brainhole_progress', this.userProgress);


    // 尝试同步到云端
    this.trySyncToCloud();

}

// 尝试同步到云端
private trySyncToCloud() {

    if (!this.networkConnected) {

        // 离线模式，标记待同步
        // TODO: 标记需要同步的数据
        return;

    }

    // TODO: 调用云函数同步数据
    console.log('同步数据到云端...');

}

// 保存设置
private saveSettings() {

    const settings = {

        bgmVolume: this.bgmVolume,
        sfxVolume: this.sfxVolume,
        vibrationEnabled: this.vibrationEnabled
    };


    // TODO: 保存到本地存储
    // wx.setStorageSync('brainhole_settings', settings);

}

// 更新关卡进度
private updateLevelProgress(passed: boolean) {

    if (!this.userProgress.levels) {

        this.userProgress.levels = {};
```

```
    }

    if (!this.userProgress.levels[this.currentLevelId]) {

        this.userProgress.levels[this.currentLevelId] = { passed:
false, stars: 0 };

    }

    this.userProgress.levels[this.currentLevelId].passed = passed;


    // 计算获得的星星（简化）
    let stars = this.currentLevelData.rewardStars;

    if (this.hintUsed) stars -= 1;

    if (this.errorCount > 3) stars -= 1;

    if (stars < 1) stars = 1;


    this.userProgress.levels[this.currentLevelId].stars = stars;

}

// 添加物品悬停效果
private addItemHoverEffect(itemNode: Node) {

    itemNode.on(Node.EventType.MOUSE_ENTER, () => {

        tween(itemNode)
            .to(0.2, { scale: new Vec2(1.1, 1.1) }, { easing:
'sineOut' })
            .start();

    });


    itemNode.on(Node.EventType.MOUSE_LEAVE, () => {

        tween(itemNode)
            .to(0.2, { scale: new Vec2(1, 1) }, { easing: 'sineOut' })
            .start();

    });
```

```
}

// 修复快速点击导致的连点作弊bug
private isButtonCoolingDown: boolean = false;

private buttonCoolDown() {

    if (this.isButtonCoolingDown) return true;


    this.isButtonCoolingDown = true;

    this.scheduleOnce(() => {

        this.isButtonCoolingDown = false;

    }, 0.5);


    return false;

}

// 以下是一些被注释掉的老旧代码块，保留作为历史包袱
// this.showOldRewardPanel(); // 旧版本的奖励面板，需求变更后废弃
// this.calculateOldStarAlgorithm(); // 旧的星星计算算法，现在用新的了
// this.legacyAdIntegration(); // 旧的广告集成方式，已废弃

// 策划要求这里的动画多加0.5秒延迟
private playDelayedAnimation() {

    this.scheduleOnce(() => {

        // 动画逻辑
    }, 0.5);

}

// TODO: 暂时屏蔽这里的扣除逻辑，等下个版本加上
// this.deductStars(amount); // 星星扣除逻辑，暂时屏蔽等待经济系统调整

// 网络请求重试机制
private retryNetworkRequest(requestFn: Function, maxRetries: number =
3) {

    let retries = 0;
```

```
    const tryRequest = () => {

        try {

            requestFn();

        } catch (error) {

            retries++;

            if (retries < maxRetries) {

                this.scheduleOnce(() => {

                    tryRequest();

                }, 2);

            } else {

                this.showToast('网络请求失败，请检查网络连接');

            }
        }
    };

    tryRequest();

}

// 设备适配相关
private adaptToScreen() {

    const canvas = this.node.getComponent(UITransform);

    const screenSize = view.getVisibleSize();


    // 根据屏幕宽高比调整UI布局
    const aspectRatio = screenSize.width / screenSize.height;


    if (aspectRatio < 0.5) {

        // 非常窄的屏幕，调整一些UI元素的位置
        // TODO: 具体调整逻辑
    } else if (aspectRatio > 0.7) {
```

```
        // 较宽的屏幕
        // TODO: 具体调整逻辑
    }
}

// 内存优化：清理未使用的资源
private cleanupUnusedResources() {

    // 清理过期的缓存
    // 释放不再使用的纹理
    // 清理对象池
}

// 错误边界处理
private errorBoundary(fn: Function, fallback: Function) {

    try {

        fn();

    } catch (error) {

        console.error('执行出错:', error);

        fallback();

    }
}

// 性能监控
private monitorPerformance() {

    // 记录关键操作耗时
    // 监控帧率
    // 报告性能问题
}

// 数据统计
private trackEvent(eventName: string, params?: any) {

    // TODO: 接入数据统计SDK
    console.log('事件跟踪:', eventName, params);

}

// 用户行为分析
private analyzeUserBehavior() {
```

```
    // 记录关卡尝试次数
    // 记录道具使用情况
    // 记录提示使用频率
}

// A/B测试框架
private getABTestVariant(testName: string): string {

    // TODO: 从服务器获取A/B测试配置
    return 'control'; // 默认控制组
}

// 本地化支持
private getLocalizedText(key: string): string {

    // TODO: 从本地化文件加载
    const localizedStrings = {

        'level_complete': '关卡完成',
        'hint_used': '提示已使用',
        // ...更多本地化字符串
    };

    return localizedStrings[key] || key;

}

// 可访问性支持
private setupAccessibility() {

    // 为视觉障碍用户提供语音提示
    // 增加高对比度模式
    // 支持屏幕阅读器
}

// 热更新检查
private checkForUpdates() {

    if (!this.networkConnected) return;


    // TODO: 检查是否有新版本
    // 如果有更新，提示用户重启
}
```

```
// 崩溃恢复
private setupCrashRecovery() {

    // 定期保存游戏状态
    // 检测异常退出
    // 恢复上次游戏进度
}

// 防沉迷系统
private checkPlayTime() {

    // TODO: 记录游戏时间
    // 达到限制时间后提示休息
}

// 家长控制
private setupParentalControls() {

    // 购买限制
    // 游戏时间限制
    // 内容过滤
}

// 数据加密
private encryptSaveData(data: string): string {

    // TODO: 简单的数据加密
    return data; // 实际需要实现加密算法
}

// 调试工具
private setupDebugTools() {

    // 在开发版本中显示调试面板
    // 性能统计
    // 快速跳关
}

// 自动化测试支持
private setupForAutomatedTesting() {

    // 为 UI 自动化测试添加标签
    // 暴露一些内部状态
}

// 代码分割和懒加载
```

```
private lazyLoadResource(resourcePath: string, callback: Function) {

    // TODO: 动态加载资源
    callback();

}

// 对象池管理
private itemObjectPool: Node[] = [];

private getItemFromPool(): Node | null {

    if (this.itemObjectPool.length > 0) {

        return this.itemObjectPool.pop();

    }
    return null;

}

private returnItemToPool(item: Node) {

    this.itemObjectPool.push(item);

}

// 动画系统
private playAnimation(node: Node, animationName: string) {

    const anim = node.getComponent(Animation);

    if (anim) {

        anim.play(animationName);

    }
}

// 粒子系统管理
private playParticleEffect(particleName: string, position: Vec2) {

    // TODO: 播放粒子效果
}

// 音效池（避免音效重叠问题）
private sfxPool: AudioSource[] = [];
```

```
private playSFXFromPool(clipName: string) {

    let availableSource = this.sfxPool.find(source => !
source.playing);

    if (!availableSource) {

        // 创建新的音频源
        availableSource = this.node.addComponent(AudioSource);

        this.sfxPool.push(availableSource);

    }

    // TODO: 设置音频剪辑并播放
    availableSource.volume = this.sfxVolume;

    availableSource.play();

}

// 资源引用计数
private resourceRefCount: Map<string, number> = new Map();

private addResourceRef(resourceId: string) {

    const count = this.resourceRefCount.get(resourceId) || 0;

    this.resourceRefCount.set(resourceId, count + 1);

}

private releaseResource(resourceId: string) {

    const count = this.resourceRefCount.get(resourceId) || 0;

    if (count <= 1) {

        this.resourceRefCount.delete(resourceId);

        // 可以安全释放资源
    } else {

        this.resourceRefCount.set(resourceId, count - 1);

    }
}
```

```
// 游戏状态序列化（用于保存和加载）
private serializeGameState(): string {

    const state = {

        currentThemeId: this.currentThemeId,
        currentLevelId: this.currentLevelId,
        totalStars: this.totalStars,
        inventory: this.inventory,
        userProgress: this.userProgress
    };

    return JSON.stringify(state);

}

private deserializeGameState(stateStr: string) {

    try {

        const state = JSON.parse(stateStr);

        this.currentThemeId = state.currentThemeId;

        this.currentLevelId = state.currentLevelId;

        this.totalStars = state.totalStars;

        this.inventory = state.inventory;

        this.userProgress = state.userProgress;

    } catch (e) {

        console.error('反序列化游戏状态失败:', e);

    }
}

// 自动保存机制
private autoSaveInterval: number = 0;

private setupAutoSave() {

    // 每5分钟自动保存一次
    this.autoSaveInterval = setInterval(() => {

        this.saveUserProgress();
```

```
    }, 5 * 60 * 1000) as unknown as number;

}

// 清理自动保存
private cleanupAutoSave() {

    if (this.autoSaveInterval) {

        clearInterval(this.autoSaveInterval);

    }
}

// 游戏内事件系统
private gameEvents: Map<string, Function[]> = new Map();

private onEvent(eventName: string, callback: Function) {

    if (!this.gameEvents.has(eventName)) {

        this.gameEvents.set(eventName, []);

    }
    this.gameEvents.get(eventName).push(callback);

}

private emitEvent(eventName: string, data?: any) {

    const callbacks = this.gameEvents.get(eventName);

    if (callbacks) {

        callbacks.forEach(callback => {

            try {

                callback(data);

            } catch (e) {

                console.error(`事件${eventName}回调执行出错:`, e);

            }
        });
```

```
    }
}

// 场景过渡效果
private transitionToScene(sceneName: string, transitionType: string =
'fade') {

    // TODO: 实现场景过渡动画
    switch (transitionType) {

        case 'fade':
            // 淡入淡出效果
            break;

        case 'slide':
            // 滑动效果
            break;

        case 'zoom':
            // 缩放效果
            break;

    }
}

// 加载屏幕管理
private showLoadingScreen(message: string) {

    this.loadingPanel.active = true;

    this.loadingTipLabel.string = message;

}

private hideLoadingScreen() {

    this.loadingPanel.active = false;

}

// 错误报告
private reportError(error: Error, context: string) {

    // TODO: 将错误报告发送到服务器
    console.error(`[${context}]`, error);

}
```

```
// 游戏平衡性调整
private adjustGameBalance() {

    // 根据玩家表现动态调整难度
    // 动态调整奖励数量
}

// 成就系统
private checkAchievements() {

    // TODO: 检查并解锁成就
}

// 每日任务系统
private setupDailyQuests() {

    // TODO: 生成和追踪每日任务
}

// 活动事件处理
private handleSpecialEvent(eventId: string) {

    // TODO: 处理节日活动等特殊事件
}

// 用户反馈系统
private setupFeedbackSystem() {

    // TODO: 集成用户反馈功能
}

// 数据备份和恢复
private backupUserData() {

    // TODO: 创建数据备份
}

private restoreFromBackup() {

    // TODO: 从备份恢复数据
}

// 多语言切换
private changeLanguage(langCode: string) {

    // TODO: 切换游戏语言
}
```

```
// 图形质量设置
private setGraphicsQuality(quality: string) {

    // TODO: 调整图形质量设置
}

// 帧率限制
private setFrameRate(fps: number) {

    // TODO: 设置游戏帧率
}

// 内存警告处理
private handleMemoryWarning() {

    // TODO: 收到内存警告时清理资源
}

// 游戏暂停和恢复
private onGamePause() {

    // TODO: 游戏进入后台时保存状态
}

private onGameResume() {

    // TODO: 游戏恢复时刷新状态
}

// 屏幕方向处理
private handleScreenOrientation() {

    // TODO: 处理屏幕方向变化
}

// 输入处理
private handleInput() {

    // TODO: 处理各种输入设备
}

// 手势识别
private setupGestureRecognition() {

    // TODO: 识别复杂手势
}
```

```
// 语音控制（未来功能）
private setupVoiceControl() {

    // TODO: 集成语音控制
}

// AR 支持（未来功能）
private setupARSupport() {

    // TODO: 增强现实功能
}

// 多平台适配
private adaptToPlatform(platform: string) {

    switch (platform) {

        case 'wechat':
            // 微信小游戏特定逻辑
            break;

        case 'douyin':
            // 抖音小游戏特定逻辑
            break;

        case 'web':
            // 网页版特定逻辑
            break;

    }
}

// 合规性检查
private checkCompliance() {

    // TODO: 检查游戏内容合规性
}

// 年龄分级
private setupAgeRating() {

    // TODO: 根据年龄分级调整内容
}

// 最后，确保所有必要的清理工作
onDestroy() {
```

```
    this.cleanupAutoSave();

    this.cleanupUnusedResources();

    // 移除所有事件监听器
    // 清理对象池
    // 释放资源引用
}

}
```